

JavaScript & TypeScript

Méthodes, asynchrone, projet TypeScript et types essentiels

push(), map(), filter(), find()

tableaux et structuredClone()

sort(), localeCompare(), reduce()

trier les nombres et les textes

fonctions fléchées

comprendre return

for...in / for...of

parcourir clés et valeurs

async / await

attendre des Promises

Promise.all()

lancer en parallèle

npm et tsconfig

initialiser un projet TS

types TypeScript

du niveau essentiel au bonus

push(), map() et filter()

Ajouter, transformer et sélectionner des valeurs

push() - modifie le tableau

Ajoute un ou plusieurs éléments à la fin du tableau existant.

CODE

```
const fruits = ["pomme", "banane"];
fruits.push("orange");
console.log(fruits);
```

Résultat : ["pomme", "banane", "orange"]

map() - nouveau tableau

Transforme chaque élément et retourne un nouveau tableau.

CODE

```
const nombres = [1, 2, 3];
const doubles = nombres.map(
  nombre => nombre * 2
);
console.log(doubles);
```

Résultat : [2, 4, 6]

filter() - nouveau tableau

Garde uniquement les éléments qui respectent une condition.

CODE

```
const ages = [12, 18, 25, 15];
const majeurs = ages.filter(
  age => age >= 18
);
console.log(majeurs);
```

Résultat : [18, 25]

find()

Trouver le premier élément qui respecte une condition

find() - chercher une valeur

find() parcourt un tableau et retourne le premier élément qui respecte la condition.

CODE

```
const nombres = [4, 7, 10, 15];  
const resultat = nombres.find(  
  nombre => nombre > 8  
);  
console.log(resultat);
```

Résultat : 10 - la recherche s'arrête au premier résultat.

Si aucun élément ne correspond

find() retourne undefined.

CODE

```
const resultat = nombres.find(nombre => nombre > 20);  
console.log(resultat); // undefined
```

find() ou filter() ?

find()

retourne le premier élément trouvé.

filter()

retourne un tableau avec tous les résultats.

Avec des objets

On peut chercher selon une propriété.

CODE

```
const trajet = trajets.find(  
  t => t.id === 2);
```

Copie par valeur, copie par référence

Primitifs copiés par valeur - objets, tableaux et fonctions copiés par référence

Primitifs - copiés par valeur

string, number, boolean, null, undefined, bigint et symbol. L'affectation crée une copie indépendante : chaque variable a sa propre valeur.

CODE

```
let a = 10;
let b = a;      // b reçoit une copie de la valeur
b = 20;
console.log(a); // 10
```

■ **Résultat : 10 - modifier b ne change jamais a.**

Objets, tableaux, fonctions - copiés par référence

L'affectation ne copie pas l'objet : elle copie la référence (l'adresse en mémoire).
Les deux variables pointent vers le même objet.

CODE

```
const original = { nom: "Alice" };
const copie = original; // même objet, deux noms
copie.nom = "Bob";
console.log(original.nom); // "Bob"
```

■ **Résultat : "Bob" - copie et original désignent le même objet.**

Même règle en paramètre

Une fonction reçoit la référence :
modifier l'objet dedans modifie
aussi l'original.

CODE

```
function renommer(p) {
  p.nom = "Chloe";
}
renommer(original);
```

À retenir

Primitif : la valeur est copiée.

Objet : la référence est partagée.

Pour une vraie copie indépendante :

```
structuredClone(objet)
```

Voir la page suivante. Le spread ...
ne fait qu'une copie superficielle.

structuredClone()

Créer une copie indépendante d'une valeur

structuredClone() - une copie profonde

structuredClone() crée une nouvelle copie d'un objet ou d'un tableau, y compris les valeurs imbriquées.

CODE

```
const original = {  
  nom: "Alice",  
  adresse: { ville: "Paris" }  
};  
  
const copie = structuredClone(original);  
copie.adresse.ville = "Lyon";
```

original.adresse.ville reste "Paris".

Pourquoi l'utiliser ?

Les modifications faites dans la copie ne changent pas l'objet original, même dans les objets et tableaux imbriqués.

La copie et l'original deviennent indépendants.

À ne pas confondre

La syntaxe ... réalise seulement une copie superficielle.

CODE

```
const copie = { ...original };
```

À retenir

Syntaxe :

structuredClone(valeur)

Copie les objets imbriqués. Ne copie pas les fonctions.

sort() et reduce()

Trier des valeurs et construire un résultat final

sort() - modifie le tableau

Pour trier des nombres, on fournit une fonction de comparaison. Avec $a - b$, le tri est croissant.

CODE

```
const nombres = [10, 3, 25, 1];
nombres.sort((a, b) => a - b);
console.log(nombres);
```

Résultat : [1, 3, 10, 25] - négatif : a avant b ; positif : b avant a.

reduce() - une seule valeur

Parcourt tout le tableau pour construire un total, une somme, un objet ou un autre résultat unique.

CODE

```
const prix = [10, 20, 5];
const total = prix.reduce(
  (accumulateur, valeur) =>
    accumulateur + valeur,
  0
);
console.log(total);
```

Résultat : 35. La valeur 0 initialise l'accumulateur.

localeCompare()

Comparer et trier des chaînes de caractères

localeCompare() - comparer du texte

Cette méthode compare deux chaînes. Elle est surtout utilisée avec `sort()` pour obtenir un tri alphabétique correct.

CODE

```
const villes = ["Lyon", "Paris", "Bordeaux"];
villes.sort((a, b) => a.localeCompare(b, "fr"));
console.log(villes);
```

Résultat : ["Bordeaux", "Lyon", "Paris"]

La valeur retournée

nombre négatif : a doit être placé avant b

0 : les deux chaînes sont équivalentes

nombre positif : a doit être placé après b

Teste < 0, === 0 ou > 0 - pas exactement -1 ou 1.

Trier de Z à A

Il suffit d'inverser a et b.

CODE

```
villes.sort((a, b) =>
  b.localeCompare(a));
```

Trier des objets

Compare la propriété qui contient le texte.

CODE

```
utilisateurs.sort((a, b) =>
  a.nom.localeCompare(b.nom));
```

Number.isInteger()

Vérifier qu'une valeur est bien un nombre entier

Number.isInteger() – entier ou pas ?

La méthode retourne true si la valeur est un nombre entier, sinon false.

CODE

```
Number.isInteger(4);      // true
Number.isInteger(4.5);    // false
Number.isInteger(4.0);    // true  (4.0 vaut exactement 4)
Number.isInteger(-12);    // true
```

Un entier négatif reste un entier ; seule la partie décimale disqualifie.

Aucune conversion automatique

Contrairement au isNaN() global, la valeur n'est jamais convertie en nombre.

CODE

```
Number.isInteger("4");    // false  (chaîne, pas nombre)
Number.isInteger(NaN);    // false
Number.isInteger(null);   // false
```

Tout ce qui n'est pas de type number retourne directement false.

Avec filter()

Garder uniquement les entiers d'un tableau.

CODE

```
const entiers =
  [1, 2.5, 3, 7.9].filter(
    n => Number.isInteger(n));
// [1, 3]
```

À ne pas confondre

Number.isInteger() vérifie sans rien modifier

Math.round() arrondit

parseInt() convertit une chaîne en entier : "42" 42

En TypeScript

Number.isInteger(x: unknown): boolean

accepte n'importe quelle valeur : pratique pour vérifier une donnée de type unknown.

Fonctions fléchées =>

Le rôle de return dépend des accolades

La règle à retenir

Avec des accolades { } : écris return.

Sans accolades : l'expression est retournée automatiquement.

Avec accolades

Le bloc peut contenir plusieurs instructions. La valeur à renvoyer doit être indiquée.

CODE

```
const doubler = (nombre) => {  
  return nombre * 2;  
};  
console.log(doubler(5));
```

Résultat : 10

Sans accolades

Quand il n'y a qu'une seule expression, son résultat est renvoyé automatiquement.

CODE

```
const doubler = nombre => nombre * 2;  
console.log(doubler(5));
```

Résultat : 10

Attention : parenthèses ≠ accolades

Les parenthèses entourent souvent les paramètres. Ce sont les accolades qui changent la règle de return.

CODE

```
(a, b) => a + b  
(a, b) => { return a + b; }
```

for...in et for...of

L'une parcourt les clés, l'autre parcourt les valeurs

for...in - les clés

À utiliser surtout avec un objet. La variable reçoit le nom de chaque propriété.

CODE

```
const personne = {  
  nom: "Lina",  
  age: 25  
};  
  
for (const cle in personne) {  
  console.log(cle);  
  console.log(personne[cle]);  
}
```

cle vaut "nom", puis "age".

for...of - les valeurs

À utiliser surtout avec un tableau. La variable reçoit directement chaque valeur.

CODE

```
const fruits = [  
  "pomme",  
  "banane",  
  "orange"  
];  
  
for (const fruit of fruits) {  
  console.log(fruit);  
}
```

fruit contient directement la valeur.

Mémo très simple

for...in = je lis les étiquettes / les clés.
for...of = je lis directement les valeurs.

Pour parcourir un tableau avec son index, utilise plutôt une boucle for classique ou forEach().

Object.entries()

Parcourir un objet sous forme de paires [clé, valeur]

Object.entries() - de l'objet au tableau

Object.entries() transforme un objet en tableau de paires [clé, valeur].

CODE

```
const stock = { pomme: 3, banane: 5 };

const paires = Object.entries(stock);
console.log(paires);
```

Résultat : `[["pomme", 3], ["banane", 5]]`

Parcourir avec for...of

La destructuration [cle, valeur] donne directement les deux parties de chaque paire.

CODE

```
for (const [cle, valeur] of Object.entries(stock)) {
  console.log(cle, valeur);
}
```

Résultat : `pomme 3 puis banane 5`

keys() et values()

Pour n'avoir que les clés
ou que les valeurs.

CODE

```
Object.keys(stock)
// ["pomme", "banane"]

Object.values(stock)
// [3, 5]
```

Enchaîner les méthodes

entries() rend un tableau :
map, filter et reduce marchent.

CODE

```
const dispos = Object.entries(stock)
  .filter(([ , n]) => n > 3)
  .map([ , cle] => cle);

// ["banane"]
```

En TypeScript

Object.entries(stock: Record<string, number>) renvoie [string, number][] : un tableau de paires typées.

Object.values()

Récupérer uniquement les valeurs d'un objet

Object.values() – de l'objet aux valeurs

Object.values() transforme un objet en tableau contenant uniquement ses valeurs.

CODE

```
const stock = { pomme: 3, banane: 5 };  
const valeurs = Object.values(stock);  
console.log(valeurs);
```

Résultat : [3, 5] – les clés sont ignorées, seules les valeurs restent.

Parcourir avec for...of

Le tableau retourné se parcourt directement, sans destructuration.

CODE

```
for (const valeur of Object.values(stock)) {  
  console.log(valeur);  
}
```

Résultat : 3 puis 5

Somme avec reduce()

values() rend un tableau :
reduce() marche directement.

CODE

```
const total =  
  Object.values(stock)  
    .reduce((t, n) => t + n, 0);  
// 8
```

values(), keys() ou entries() ?

values() les valeurs [3, 5]

keys() les clés
["pomme", "banane"]

entries() les paires
[["pomme", 3], ["banane", 5]]

En TypeScript

Object.values(stock: Record<string, number>)
renvoie number[] : un tableau qui contient uniquement les valeurs typées.

async et await

Attendre une opération sans bloquer tout le programme

L'idée générale

Certaines opérations prennent du temps : récupérer des données, lire un fichier ou attendre une réponse. `async` et `await` rendent ce code plus facile à lire.

async

Se place avant une fonction. Une fonction `async` retourne toujours une Promise.

CODE

```
async function obtenirNombre() {  
  return 42;  
}  
obtenirNombre().then(console.log);
```

Résultat : 42

await

Attend qu'une Promise se termine, puis récupère directement son résultat.

CODE

```
async function afficher() {  
  const resultat =  
    await Promise.resolve("OK");  
  console.log(resultat);  
}  
afficher();
```

Résultat : OK

Exemple avec `fetch()` et `try...catch`

`fetch()` retourne une Promise. `try...catch` permet de gérer l'échec sans arrêter brutalement le programme.

CODE

```
async function chargerUtilisateur() {  
  try {  
    const reponse = await fetch(url);  
    const utilisateur = await reponse.json();  
    console.log(utilisateur.nom);  
  } catch (erreur) {  
    console.log("Erreur de chargement");  
  }  
}  
chargerUtilisateur();
```

Ordre : lancer la demande -> attendre -> convertir les données -> utiliser le résultat.

Les Promises

Une valeur qui sera disponible plus tard

L'idée générale

Une Promise représente un résultat que JavaScript n'a pas encore. Elle peut réussir ou échouer plus tard.

1 pending

opération en cours

2 fulfilled

opération réussie

3 rejected

opération échouée

Le cycle d'une Promise

pending devient ensuite fulfilled ou rejected. Une fois terminée, la Promise ne change plus.

CODE

```
pending -> fulfilled  
pending -> rejected
```

Exemple simple

Au départ, la Promise est en attente. Après 100 ms, `resolve()` la fait réussir.

CODE

```
const promesse = new Promise((resolve) => {  
  setTimeout(() => {  
    resolve("Trajet trouvé");  
  }, 100);  
});  
  
promesse.then(console.log)  
  .catch(console.error);
```

then() utilise la valeur réussie ; catch() gère une Promise rejetée.

Promise.all()

Lancer plusieurs Promises en même temps

À quoi ça sert ?

`Promise.all()` lance plusieurs opérations ensemble et attend qu'elles soient toutes terminées.

Exemple avec le trajet

CODE

```
const [trajet, passagers] = await Promise.all([
  getTrajet(id),
  getPassagers(id)
]);
```

Les résultats gardent le même ordre que les Promises du tableau.

L'un après l'autre

Le deuxième appel démarre seulement après la fin du premier.

CODE

```
const trajet = await getTrajet(id);
const passagers =
  await getPassagers(id);
```

Environ 200 ms

En parallèle

Les deux appels démarrent immédiatement.

CODE

```
const resultats = await Promise.all([
  getTrajet(id),
  getPassagers(id)
]);
```

Environ 100 ms

À retenir

1. Utilise `Promise.all()` quand les opérations ne dépendent pas les unes des autres.
2. `await` attend tous les résultats.
3. Si une Promise est rejetée, `Promise.all()` est aussi rejetée.

Le trajet du code

Qui écrit, qui compile et qui exécute ?

L'idée principale

Tu écris du TypeScript dans des fichiers `.ts`. Le compilateur `tsc` vérifie les types et produit du JavaScript dans des fichiers `.js`. Ensuite, Node.js exécute le JavaScript.

CODE

```
src/index.ts -> npx tsc -> dist/index.js
               -> node dist/index.js
```

TypeScript sert à écrire et vérifier. Node.js exécute le JavaScript produit.

npm

Installe des paquets et tient à jour `package.json`.

CODE

```
npm install ...
```

npx

Lance un outil installé dans le projet, par exemple `tsc`.

CODE

```
npx tsc
```

tsc

C'est le compilateur TypeScript. Il transforme le code `.ts` en code `.js`.

CODE

```
npx tsc
```

Node.js

Il exécute le JavaScript final, généralement dans le dossier `dist`.

CODE

```
node dist/index.js
```

npm init

Transformer le dossier courant en projet npm

La commande, mot par mot

npm est le gestionnaire de paquets. init veut dire « initialiser ». La commande prépare le dossier courant comme un projet npm.

CODE

```
npm init
```

Ce qui se passe sans option

npm pose plusieurs questions : nom du projet, version, description, fichier principal, auteur, licence, etc. Tu peux appuyer sur Entrée pour accepter la valeur proposée.

CODE

```
package name: (mon-projet)
version: (1.0.0)
description:
entry point: (index.js)
author:
license: (ISC)
```

La version rapide : npm init -y

-y signifie « yes ». npm accepte automatiquement les valeurs par défaut et crée directement le fichier `package.json`.

CODE

```
npm init -y
```

npm init ne télécharge pas TypeScript et ne compile aucun fichier.

package.json

La fiche d'identité et la liste des paquets

Le fichier créé par npm init

Il décrit le projet. npm y ajoute ensuite les paquets installés, les commandes de projet et quelques informations générales.

CODE

```
{
  "name": "mon-projet",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {},
  "license": "ISC"
}
```

main

Indique un fichier principal pour certains usages npm. Cela ne signifie pas que TypeScript va compiler uniquement ce fichier.

CODE

```
"main": "index.js"
```

scripts

Permet de donner un nom court à une commande souvent utilisée.

CODE

```
"scripts": {
  "build": "tsc"
}
```

Ce que package.json ne fait pas

Il ne contient pas ton code, ne crée pas le dossier src, ne compile rien et ne lance rien tout seul. C'est seulement un fichier de configuration pour npm.

À retenir : package.json décrit le projet ; tsconfig.json règle le compilateur TypeScript.

Installer TypeScript

Comprendre `npm install --save-dev typescript`

La commande, morceau par morceau

`npm` appelle le gestionnaire de paquets. `install` télécharge et ajoute un paquet. `--save-dev` le classe comme outil de développement. `typescript` est le paquet demandé.

CODE

```
npm install --save-dev typescript
# forme courte équivalente :
npm install -D typescript
```

1 node_modules

fichiers téléchargés

2 package.json

typescript ajouté

3 package-lock

versions exactes

Les trois changements produits

1. `node_modules/` reçoit les fichiers du paquet.
2. `package.json` reçoit TypeScript dans `devDependencies`.
3. `package-lock.json` mémorise précisément les versions installées.

Tu ne modifies normalement pas `node_modules` à la main.

dependencies et devDependencies

Ce qui sert à l'application et ce qui sert à la fabriquer

dependencies

Paquets dont l'application a besoin pendant qu'elle fonctionne. Exemple : une bibliothèque réellement utilisée par le code exécuté.

CODE

```
npm install express
```

devDependencies

Outils surtout utiles pour développer, compiler, vérifier ou tester le projet.

CODE

```
npm install -D typescript
```

Exemple dans package.json

CODE

```
{
  "dependencies": {
    "express": "..."
  },
  "devDependencies": {
    "typescript": "..."
  }
}
```

Et en production ?

On peut choisir de ne pas installer les outils de développement avec `--omit=dev`. Le serveur reçoit alors les dependencies, mais pas les devDependencies.

CODE

```
npm install --omit=dev
```

devDependencies ne veut pas dire « supprimées automatiquement » : c'est la commande d'installation qui décide.

npx tsc --init

Créer la configuration du compilateur

La commande, mot par mot

npx cherche un outil installé dans le projet et le lance. tsc est le compilateur TypeScript. --init lui demande d'initialiser sa configuration.

CODE

```
npx tsc --init
```

Le résultat exact

La commande crée le fichier tsconfig.json dans le dossier courant. Ce fichier contient les règles que tsc utilisera plus tard pour vérifier et compiler le projet.

CODE

```
mon-projet/  
|-- node_modules/  
|-- package.json  
|-- package-lock.json  
`-- tsconfig.json
```

Ce que --init ne fait pas

La commande ne crée pas automatiquement src ou dist, ne fabrique aucun fichier JavaScript et n'exécute pas le programme.

--init prépare les réglages. La compilation arrive plus tard avec npx tsc.

tsconfig.json

Les règles utilisées par le compilateur TypeScript

Un exemple simple

Le fichier est au format JSON. La partie `compilerOptions` contient les principaux réglages du compilateur.

CODE

```
{
  "compilerOptions": {
    "rootDir": "./src",
    "outDir": "./dist",
    "strict": true,
    "target": "ES2022",
    "module": "CommonJS"
  }
}
```

rootDir / outDir

`rootDir` indique où se trouve ton TypeScript. `outDir` indique où placer le JavaScript généré.

CODE

```
src/index.ts
->
dist/index.js
```

strict

Active des vérifications plus exigeantes, notamment autour de `null`, `undefined` et des paramètres mal typés.

CODE

```
"strict": true
```

target et module

`target` choisit la version de JavaScript produite. `module` choisit la manière dont les imports et exports sont transformés. Ces valeurs dépendent de l'environnement du projet.

tsconfig.json règle la compilation ; il ne contient pas ton code source.

De index.ts à index.js

Créer le code, le compiler, puis l'exécuter

1 mkdir src

créer le dossier

2 index.ts

écrire le code

3 npx tsc

compiler

4 node ...js

exécuter

1. Écrire du TypeScript

CODE

```
mkdir src

# fichier src/index.ts
const message: string = "Bonjour";
console.log(message);
```

2. Compiler

tsc lit tsconfig.json, vérifie les types et produit le JavaScript.

CODE

```
npx tsc
```

3. Exécuter

Node.js ouvre le fichier JavaScript produit dans le dossier dist.

CODE

```
node dist/index.js
```

Important

Compiler ne veut pas dire exécuter. Après `npx tsc`, le programme n'a pas encore été lancé.

npx tsc = fabriquer le JavaScript ; node dist/index.js = lancer le programme.

Initialiser un projet complet

La séquence minimale et le résultat de chaque commande

Les commandes à recopier

CODE

```
mkdir mon-projet
cd mon-projet
npm init -y
npm install -D typescript
npx tsc --init
mkdir src
# créer src/index.ts
npx tsc
node dist/index.js
```

L'arborescence finale

CODE

```
mon-projet/
|-- src/
|   |-- index.ts
|-- dist/
|   |-- index.js
|-- node_modules/
|-- package.json
|-- package-lock.json
`-- tsconfig.json
```

Mode automatique

Le compilateur reste ouvert et recompilera après chaque sauvegarde.

CODE

```
npx tsc --watch
```

Trois confusions

npm init crée le projet npm ; --init crée la configuration TS ; tsc compile.

Développement et production

Où se trouve le compilateur TypeScript ?

Mode développement

Tu écris les fichiers .ts, tu compiles, tu testes et tu corriges. TypeScript se trouve généralement dans devDependencies.

CODE

```
npm install -D typescript
npx tsc
```

Mode production

Les utilisateurs utilisent l'application. Node.js exécute généralement les fichiers .js déjà produits.

CODE

```
npm ci --omit=dev
node dist/index.js
```

Le trajet du code

CODE

```
src/index.ts
  | compilation avec tsc
  v
dist/index.js
  | exécution avec Node.js
  v
programme en fonctionnement
```

La nuance du déploiement

Certaines plateformes reçoivent le TypeScript et compilent pendant le déploiement. Dans ce cas, le compilateur est présent pendant la phase de construction, puis l'application finale exécute le JavaScript généré.

En production, il n'y a généralement plus besoin de tsc au moment où le programme tourne.

Types de base et annotations

Typer une variable, un paramètre et un retour

Les types de base

string représente du texte, number un nombre et boolean une valeur vraie ou fausse.

CODE

```
const conducteur: string = "Alice";  
const prix: number = 15;  
const complet: boolean = false;
```

Annoter une fonction

Chaque paramètre reçoit un type après les deux-points. Le type placé après les parenthèses indique la valeur retournée par la fonction.

CODE

```
function recette(  
  passagers: number,  
  prix: number  
): number {  
  return passagers * prix;  
}
```

Type explicite ou type deviné ?

TypeScript comprend automatiquement que `const prix = 15` est un nombre. Les annotations sont particulièrement utiles pour les paramètres, les retours et les structures de données partagées.

L'objectif n'est pas d'écrire des types partout, mais de rendre le contrat du code clair.

Tableaux et tableaux d'objets

Comprendre T[], Array<T> et Trajet[]

Deux écritures identiques

number[] et Array<number> signifient toutes les deux « tableau de nombres ».

CODE

```
const prix: number[] = [10, 20];
const autres: Array<number> = [5, 8];
```

Un tableau de chaînes

Le type placé avant [] indique ce que le tableau peut contenir.

CODE

```
const noms: string[] = [
  "Alice",
  "Bob"
];
```

Un tableau d'objets Trajet

Trajet[] signifie que chaque élément doit respecter la forme décrite par Trajet.

CODE

```
interface Trajet {
  id: number;
  conducteur: string;
  prix: number;
}

const trajets: Trajet[] = [
  { id: 1, conducteur: "Alice", prix: 15 }
];
```

Erreur détectée avant l'exécution

Si un élément oublie prix ou met du texte dans id, TypeScript signale le problème avant que le programme soit lancé.

interface ou type ?

Deux manières de nommer une structure

interface

Très naturelle pour décrire la forme d'un objet. Elle peut être étendue avec `extends` et peut être ouverte.

CODE

```
interface Trajet {  
  id: number;  
  conducteur: string;  
}
```

type

Peut décrire un objet, mais aussi une union, un tuple, un tableau ou d'autres combinaisons de types.

CODE

```
type Trajet = {  
  id: number;  
  conducteur: string;  
};
```

En pratique

Pour un simple objet, `interface` et `type` sont presque interchangeables. Une réponse simple en entretien : `interface` pour les objets, `type` pour les autres compositions.

L'important est surtout de savoir lire les deux formes et de rester cohérent dans un projet.

Exemple avec extends

CODE

```
interface TrajetPremium extends Trajet {  
  wifi: boolean;  
}
```

Unions de littéraux

Autoriser seulement certaines valeurs

Limiter un statut

Le symbole `|` signifie « ou ». La variable ne peut prendre que l'une des chaînes indiquées.

CODE

```
type Statut =  
  | "confirmée"  
  | "annulée"  
  | "en_attente";  
  
let statut: Statut = "confirmée";
```

Dans un objet réel

CODE

```
interface Reservation {  
  id: number;  
  statut: "confirmée" | "annulée";  
}  
  
const r: Reservation = {  
  id: 1,  
  statut: "confirmée"  
};
```

enum ou union ?

Les deux existent. Pour de simples valeurs textuelles, les unions littérales sont souvent préférées : elles sont légères, lisibles et ne créent pas d'objet JavaScript supplémentaire.

Une faute comme "confirme" au lieu de "confirmée" est refusée par le compilateur.

Optionnel, null et undefined

Pourquoi le mode strict oblige à vérifier

Une propriété optionnelle

Le point d'interrogation signifie que la propriété peut être absente. Son type réel devient donc `string | undefined`.

CODE

```
interface Passager {
  nom: string;
  email?: string;
}
```

undefined

Une valeur manque ou une propriété n'existe pas.

CODE

```
const email = passager.email;
// string | undefined
```

null

Une absence choisie et explicite, souvent utilisée comme résultat possible.

CODE

```
const trajet: Trajet | null = null;
```

Vérifier avec un if

Après la condition, TypeScript comprend que la valeur n'est plus undefined ou null.

CODE

```
if (passager.email !== undefined) {
  console.log(passager.email.toLowerCase());
}

if (trajet !== null) {
  console.log(trajet.conducteur);
}
```

En mode strict, le compilateur t'empêche d'utiliser une valeur potentiellement absente sans vérification.

? . et ??

Lire une valeur optionnelle et prévoir une valeur par défaut

Optional chaining : ?.

Si la valeur située à gauche est null ou undefined, l'expression renvoie undefined au lieu de provoquer une erreur.

CODE

```
const nom = trajet?.conducteur;
// string | undefined
```

Nullish coalescing : ??

Si la valeur de gauche est null ou undefined, TypeScript utilise la valeur de droite.

CODE

```
const conducteur =
  trajet?.conducteur ?? "Inconnu";
```

? . ne remplace pas toujours if

Utilise ? . quand tu veux seulement lire une valeur sans erreur.

CODE

```
trajet?.conducteur
```

if pour faire plusieurs actions

Utilise une condition lorsque tu dois exécuter un bloc entier seulement si la valeur existe.

CODE

```
if (trajet) {
  afficher(trajet);
}
```

À ne pas confondre avec ||

?? réagit seulement à null et undefined. La valeur 0, la chaîne vide et false restent valides.

Typer une fonction async

Comprendre Promise<Trajet | null>

Lire la signature

id: number décrit le paramètre. Promise<Trajet | null> signifie que la fonction donnera plus tard soit un trajet, soit null.

CODE

```
async function trouverTrajet(
  id: number
): Promise<Trajet | null>
```

Exemple complet

CODE

```
async function trouverTrajet(
  id: number
): Promise<Trajet | null> {
  try {
    return await getTrajet(id);
  } catch (err: unknown) {
    if (err instanceof Error) {
      console.log(err.message);
    }
    return null;
  }
}
```

Après await

await retire l'enveloppe Promise. La variable devient Trajet | null.

CODE

```
const trajet =
  await trouverTrajet(1);
```

Une fonction async

Même si elle retourne une valeur simple, une fonction async retourne toujours une Promise.

CODE

```
async function f(): Promise<number> {
  return 42;
}
```

Record et génériques

Objets-dictionnaires et types réutilisables

Record

Cela décrit un objet dont les clés sont des chaînes et dont les valeurs sont des nombres. C'est pratique pour construire des totaux par conducteur ou par catégorie.

CODE

```
const recettes: Record<string, number> = {
  Alice: 50,
  Bob: 46
};

recettes["Chloe"] = 24;
```

Un générique simple

T est une case vide pour un type. Au moment de l'appel, TypeScript détermine si T représente une chaîne, un nombre, un trajet, etc.

CODE

```
function premier<T>(tab: T[]): T | undefined {
  return tab[0];
}

const nom = premier(["Alice", "Bob"]);
// string | undefined
```

Array

Array<Trajet> signifie « tableau de Trajet ». C'est la même chose que Trajet[].

Promise

Promise<Trajet> signifie « promesse qui donnera plus tard un Trajet ».

Les types utilitaires

Partial, Pick, Omit et Readonly

Le type de départ

CODE

```
interface Trajet {
  id: number;
  conducteur: string;
  depart: string;
  arrivee: string;
  prix: number;
}
```

Partial

Toutes les propriétés deviennent optionnelles.

CODE

```
const modif: Partial<Trajet> = {
  prix: 20
};
```

Pick

Conserve uniquement les propriétés choisies.

CODE

```
type Itineraire = Pick<Trajet,
  "depart" | "arrivee">;
```

Omit

Conserve toutes les propriétés sauf celles retirées.

CODE

```
type Public = Omit<Trajet,
  "prix">;
```

Readonly

Empêche de réassigner les propriétés dans le code TypeScript.

CODE

```
const t: Readonly<Trajet> = trajet;
```

À retenir

Ces outils transforment seulement les types. Ils n'ajoutent aucune donnée et ne modifient aucun objet pendant l'exécution.

Narrowing et catch

TypeScript affine le type après une vérification

Narrowing avec typeof

Avant le `if`, `x` peut avoir deux types. Dans chaque branche, TypeScript réduit les possibilités.

CODE

```
function afficher(x: string | number) {
  if (typeof x === "string") {
    console.log(x.toUpperCase());
  } else {
    console.log(x.toFixed(2));
  }
}
```

Le même principe avec null

CODE

```
const trajet: Trajet | null =
  await trouverTrajet(1);

if (trajet !== null) {
  console.log(trajet.conducteur);
}
```

Dans catch, err est unknown

JavaScript peut rejeter n'importe quelle valeur. Il faut donc vérifier que `err` est réellement un objet `Error` avant de lire `message`.

CODE

```
try {
  await getTrajet(1);
} catch (err: unknown) {
  if (err instanceof Error) {
    console.log(err.message);
  }
}
```

unknown, any, as et type guard

Vérifier sans désactiver TypeScript

any

Désactive presque toutes les vérifications. Une erreur peut alors n'apparaître qu'au moment de l'exécution.

CODE

```
let valeur: any = 42;
valeur.inexistant();
```

unknown

Le type est encore inconnu. Tu dois vérifier avant d'utiliser la valeur.

CODE

```
let valeur: unknown = 42;
if (typeof valeur === "number") {
  console.log(valeur + 1);
}
```

Un type guard personnalisé

Le retour `x is Trajet` signifie : si la fonction retourne `true`, TypeScript peut considérer que `x` est un `Trajet`.

CODE

```
function estTrajet(x: unknown): x is Trajet {
  if (typeof x !== "object" || x === null) {
    return false;
  }
  const objet = x as Record<string, unknown>;
  return typeof objet.id === "number";
}
```

Assertion de type : as

`as` ne transforme pas et ne vérifie pas la donnée. Tu demandes seulement à TypeScript de te croire. Une mauvaise assertion peut cacher un bug.

CODE

```
const trajet = donnee as Trajet;
```

Bonne réponse : préférer `unknown` + vérification ; éviter `any` ; utiliser `as` avec prudence.

readonly et as const

Empêcher certaines modifications et conserver des valeurs précises

readonly

Empêche une réaffectation dans le code TypeScript. Cela protège l'intention, mais ne gèle pas forcément l'objet à l'exécution.

CODE

```
interface Trajet {
  readonly id: number;
  conducteur: string;
}
trajet.id = 2; // erreur TS
```

as const

Conserve les valeurs littérales précises et rend les propriétés en lecture seule.

CODE

```
const reservation = {
  statut: "confirmée",
  places: 2
} as const;
```

Créer une union depuis un tableau constant

CODE

```
const STATUTS = [
  "confirmée",
  "annulée",
  "en_attente"
] as const;

type Statut = typeof STATUTS[number];
```

Checklist à maîtriser

Essentiel : types de base, tableaux, annotations, interface/type, unions, propriétés optionnelles, null/undefined, `?.`, `??`, fonctions async, `Promise<T>`, `Record`.

Très probable : génériques, `Partial`, `Pick`, `Omit`, narrowing et erreurs unknown.

Bonus : type guards, assertions `as`, unknown contre any, `readonly` et `as const`.

interface vs type — la version complète

Tout ce qui peut te faire relancer en entretien

Declaration merging

Deux interfaces du MÊME nom fusionnent automatiquement. type ne peut pas : deux type identiques = erreur.

CODE

```
interface User { name: string }
interface User { age: number }
// User a name ET age

type User = { name: string }
type User = { age: number } // Duplicate id
```

type fait plus de choses

interface = formes d'objets. type gère aussi unions, tuples et types calculés. L'inverse n'est presque pas vrai.

CODE

```
type Statut = "on" | "off" // union
type Point = [number, number] // tuple
type Keys = keyof User // mapped
type Nullable<T> = T | null
```

Syntaxe : = ou bloc

type est un alias -> on assigne, donc =.
interface est une déclaration de bloc,
comme class ou function, donc pas de =.

CODE

```
type User = { name: string }; // avec =
interface User { name: string } // bloc

class User { } // pas de =
function f() { } // pas de =
```

Héritage : extends vs &

Les deux savent étendre, syntaxe différente. interface avec extends, type avec une intersection &.

CODE

```
interface Admin extends User {
  role: string
}

type Admin = User & { role: string }
```

Le cas qui tombe : augmenter un type externe

Le merge sert surtout à ouvrir un type que tu ne contrôles pas (une lib, l'objet global Window). Impossible avec type. C'est LA vraie utilité du declaration merging à citer en entretien.

CODE

```
interface Window {
  myApp: { version: string }
}
window.myApp.version // TS content, pas d'erreur
```

La phrase à ressortir

interface = objets + contrats de classe (implements) + merge. type = dès que tu as unions, tuples ou manipulation de types. Règle courte : interface par défaut, type quand interface ne peut pas.